子任务 2：P4 代码智能体

*目前仅考虑英文数据吧：

	任务	子任务	人选	备注
目标 1：数据集构建	无监督	P4 代码收集：以项目为单位爬取（仅考虑项目内包含.p4 文件的完整项目） *项目之间文件重复咋办？(starcoder、stackv2 如何处理？)	谢国锐	参考 1.1 的“P4 代码：”段落 ,要了解 stackv2 数据集的形式以及 bigquery 的 github 数据库
		Github 的 P4 相关 issue 和 PR 收集		参考 1. 1 的 “ Github issue/PR：” 段落，了解 stackv2 数据集
		P4 文档收集：保存所有访问的 url , 以及 url 对应的文字		参考 1.1 的“P4 文档：”段落，了解 stackv2 数据集
		StackOverflow 的 P4 问答收集		参考 1. 1 的“StackOverflow：”段落，了解 stackv2 数据集
		数据增强：随机生成 P4 和控制面代码对（ BMv2、eBPF、Tofino、FPGA Vitis P4 ）	宗珮媛	参考 1.1 的“数据增强：”段落，了解 Gauntlet、P4-testgen 等论文及其配套代码

	有监督	BMv2 环境搭建+出题		参考 1.2
		eBPF 环境搭建+出题		
		Tofino 环境搭建+出题	耿悦	
		FPGA 环境搭建+出题 Vitis Networking P4		
目标 2：模型适配		基于 LlaMA Factory，编写 Qwen 的领域化预训练 整合有监督的 4 个环境为一个沙箱，完成智能体的 function calling 功能	谢 国锐	需要调研相关论文 参考 1.4，需要 1 人

1、持续预训练获得 P4 基模型

1.1 无监督训练集构建（仅供参考 未必就按这个）

*参考《StarCoder 2 and The Stack v2: The Next Generation》

- P4 代码 :**1)从 github 下载所有包含.p4 后缀的项目(运用 bigquery 的 github 数据集，得到 github 项目名称，再去爬取 github ）；3)** https://github.com/p4lang/下的所有项目的最新版本；**4)** Tofino SDE+example 代码库。**5)** Vitis P4 IP+example 代码库

  **4)和 5)是离线的，我后面提供；需要一个完整项目 or 目录作为一个样本，这样才有概率包括控制面代码（翻阅 starcoder 细节按一致的文本存储模板，50%概率才有红色部分）：*

  <repo_name>reponame<file_sep>filepath1\ncode1<file_sep>filepath2\ncode2 ... <|endoftext|>.

- P4文档：**1)** https://p4.org/页面及其每一个子链接页面（ developer day、working group、specification 语言规范(需包括 P4、PSA、TNA、P4runtime 等)、blogs、Forum 问答社区、publication 等等子链接), 维护一个队列，爬取队列头部链接保存为一个样本，其包含的每一个子链接入队等待爬取；苏黎世网络课程；**2)**https://adv-net.ethz.ch/2018 到 2025(换年份即可下载 material 页面有 ppt)；**3)** https://github.com/p4lang/的每个项目的wiki等链接（不是仓库里的内容）；**4)** P4.org 领英的文章、文档两个模块；**5)**谷歌关键词搜索博客、文字、网站：P4、BMv2、Tofino、Vitis Networking P4、P4 eBPF ，特别是 Tofino 和 Vitis 也要搜索一些类似 user guide、IP 手册形式的 pdf；~~**6)** 微信公众号云深知网络的推文；~~**7)**

RFC 791 - 2000、8321、9817、8926、7348、768、9293、8200、7149、7426；

**HTML最终只需要相关的文本（大量抓取的网络文本含有过多的垃圾字符，如 <script>， ，ads），通过 trafilatura 库转为 XML 再转为 markdown，pdf 使用 pdftotext 库*

- P4 Github issue：https://github.com/p4lang/的每个项目的 issue；搜索其他 P4 相关项目的issue，using the language filter available in the public API（https://docs.github.com/en/search-github/github-code-search/understanding-github-code-search-syntax#language-qualifier）
- P4 Github PR :https://github.com/p4lang/的每个项目的 PR(确认下 stackv2 是否包含2023年9月前的PR 是的话 这里只需9月后的) 搜索其他P4相关项目的 PR，using the language filter available in the public API（https://docs.github.com/en/search-github/github-code-search/understanding-github-code-search-syntax#language-qualifier）
- P4 StackOverflow 问答： 关键词 P4、BMv2、Tofino、Vitis P4、P4 eBPF 等等，有StackOverflow dump数据集（StackExchange Archive, 2024. URL https://archive.org/details/stackexchange）可参考
- 数据增强：~~1) 我们构建双语数据集，上述内容大部分是英文的，要翻译为中文（借助大模型）；公众号、issue、谷歌搜索的部分内容若为中文，也要对应翻译（暂不考虑）~~；2)通过 P4 语法分析 AST ,并利用同义词库/LLM 替换变量名、函数名，注入死代码，添加多余的空格/换行等；~~3) 随机挑选 P4 代码样本（50%）做 FIM（暂不考虑）~~；4) 使用 AST (语法树) 局部变异技术，对这些种子进行小手术。比如：把 NAT 里面的某个变量名改掉、把表的大小改掉、随机在 Ingress 里塞一个不影响大局的计数器（Counter）或者测量仪（Meter）5) Gauntlet + P4-testgen 生成 P4 和控制面代码对。
- 总目标是收集/合成 10GB 左右原始数据？（TB 级能否达到）

1.2 初始监督数据集构建 -- 代码

暂定，随着训练可能会调整，前期收集可以慢些

以BMv2为例(写300题),每个样例格式如下(不光是写问题prompt ,也要给出p4、test 答案) ：

easy_task1.1

├── run.py

├── prompt/

│ └── ~~cn.txt（申文）~~

```
│       └── en.txt （英文）
├── p4src/    <-- 开启版本控制，当用 run 脚本报错时，commit 要带上报错信息
│       └──其他 .p4 文件（可选，被主文件引用的）
│       └── main.p4
├── control_p4runtime/              <-- 开启版本控制
│       └── test.py                 <-- 方案 A (P4Runtime-shell API 模式)
└── control_cli/            <-- 开启版本控制
        └── test.py         <-- 方案 B (Thrift stdin 注入模式
```

prompt 里是中文或英文指令，样例为：

```
你是一位 P4 网络专家。请针对以下题目和指定的控制面模式，输出数据面（main.p4）和控制面/测试面（test.py）。

【输出代码要求】
1. `main.p4`：完整的 P4_16 v1model 数据面源码。
2. `test.py`：仅包含以下两个标准的函数定义：
    def setup_rules():
        """
        负责流表规则下发。
        - 根据数据面类型（bmv2/tofino 等），使用合法的方式，如 p4runtime_sh 连接 localhost:9559，或 subprocess 连入 simple_switch_CLI --thrift-port 9090 等下发流表。
        - 若认为本题无需规则下发，可留空
        """
        # 你的规则下发代码（无需自行捕获异常）

    def verify_traffic():
        """
        负责流量验证。
        - 使用 Scapy 在对应的 veth 网口上进行异步收发包。
        - 若数据面涉及 register/meter/counter 等修改，则此处可调用控制面 API 检查 register/meter/counter 等的状态。
        - 必须使用显式的 `assert` 语句进行判定，且 assert 必须带有可读的错误描述字符串。
        """
        # 你的 Scapy 发包与捕获断言代码

【当前题目】
BMv2 数据面将源 IP 为 10.0.0.1 的包从 Port 2 转发出去，其余丢弃。
```

run.py 负责编译测试 p4，启动 simple_switch_grpc 交换机，下发规则测试，发送流量测试。三种测试的源码都是大模型提供，run 的样例为：

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os
import subprocess
import time
import sys
import traceback

PORTS = [0, 1, 2, 4, ..., 31]

if os.geteuid() != 0:
    print("    错误: 请使用 sudo 运行此总控脚本！")
    sys.exit(255)

def run_cmd(cmd):
    return subprocess.run(cmd, shell=True, stdout=subprocess.DEVNULL, s

def cleanup():
    os.system("pkill -f simple_switch_grpc 2>/dev/null")
    for port in PORTS:
        os.system(f"ip link delete veth{port} 2>/dev/null")
    time.sleep(0.5)

def setup_env():
    for port in PORTS:
        run_cmd(f"ip link add veth{port} type veth peer name veth{port}_sw
        run_cmd(f"ip link set dev veth{port} up")
        run_cmd(f"ip link set dev veth{port}_sw up")

def main(control_dir="control_p4runtime"):
    cleanup()
    setup_env()
```

请先把 run 改写正确后，再进行题目编写。Tofino 时（tofino_model，300 题），控制面相应改为BF-Runtime、BfShell，run脚本需要对应修改；eBPF（300题） 控制面 P4Runtime ;Vitis P4（FPGA ,300 题），控制面核心是 Runtime Driver API (C/C++ 库)。在纯软件模型中（Xilinx P4 Behavioral Model），它提供了一套外部 API（通常通过 Python 的 ctypes 加载）；

出题可以从这些里参考：BMv2 tutorial（https: //github. com/p4lang/tutorials/tree/master/exercises），BMv2 P4-learning（example https://github.com/nsg-ethz/p4-learning/tree/master/examples 和 exercise https://github.com/nsg-ethz/p4-learning/tree/master/exercises 两个文件夹），Tofino example（https://github.com/p4lang/open-p4studio/tree/main/pkgsrc/p4-examples/p4_16_programs），vitis example 目录的例子，ebpf 样例（https://github.com/p4lang/p4c/tree/main/backends/ebpf/tests/p4testdata）；注意：1、题目不要涉及复杂拓扑；2、题目应当和上述链接有异同；3、每个题目需要验证正确性，同时用 git 版本控制留下报错信息用于后续大模型调试能力训练；4、题目多样性：可以是描述意图+P4 挖空文件，让它填空+写 test.py；不同平台（BMv2、Tofino、FPGA）之间需要有 30%题目是重叠的，实现同一种功能，保持题号对应一样；同一平台内，也允许多个题目（说法不同），但要实现的功能是一样的（30%左右），题号 1.x 表示 x 道题都实现的 1 功能。可以借助大模型出题，但必须人工复检，分析是否满足题目的功能要求，出题原则为：单个交换机，需要收发包测试，保证 40%题目有控制面，功能涵盖基础 P4 用法（包头修改、表规则匹配，加减位移等等），Registers/Counters/Meters，以及 Clone/Recirculate/Resubmit 等高级功能；

宗博还有一个任务是，看一下是否有自动代码生成+测试的方法，Gauntlet+P4testgen，是否可以生成大量语料？

玩法一：利用 Gauntlet 的 `Bludgeon` 批量生产“合法 P4 语法树语料”

大模型在训练初期（预训练阶段），最大的痛点是**不知道 P4-16 的语法长什么样**（网上开源的 P4 代码太少了）。如果直接去 GitHub 爬，能找到的代码寥寥无几。

- **如何操作**：`Gauntlet` 内部用来轰炸编译器的组件叫 `Bludgeon`。它不是一个黑盒，它是一个纯粹的 **P4 代码随机生成器**。你可以直接单独调用它，让它给你生成 10 万份结构各异、充满了复杂的嵌套表达式、控制流、各种元数据操作的 `main.p4` 源码。
- **对语料的补充**：这些代码虽然毫无实际业务意义（可能只是一堆随机加减乘除和条件跳转），但它们在**词法（Lexical）和语法（Syntax）上是 100% 绝对合法且极度复杂的 P4-16 代码**。
- **训练价值**：把这 10 万份代码喂给你的基座大模型做预训练（Next Token Prediction），大模型对 P4 语言的 **Token 概率分布、关键字组合（如 `control` 、 `action` 、 `apply` ）以及语法边界** 的理解会瞬间达到专家级别，彻底解决生成代码时常出现的“括号不匹配、少写分号、变量未定义”等低级幻觉。

下方这种是低级语义，就是很细节地说明需求，testgen 路径、触发什么动作这种

🎯 玩法二：利用 P4-testgen 生产“带语义标签的指令微调语料（Instruct Data）”

预训练完了，模型只会续写代码，依然不知道用户的“业务需求”是什么。这时候你需要 **“输入 → 输出”的指令微调对（QA Pairs）**。`P4-testgen` 此时就是最完美的**自动打标机**。

- **如何操作**：
  1. 你把上面 `Bludgeon` 生成的，或者你从 GitHub 爬来的真实 `main.p4` 代码喂给 `P4-testgen`。
  2. `P4-testgen` 会针对这份代码吐出完备的测试用例。比如它分析出：“当输入报文 A 满足 `IPv4.dst == 10.0.0.1` 且控制面下发表项 B 时，交换机会执行 Action C，并将报文从端口 2 吐出。”
- **对语料的补充**：你可以用一个简单的 Python 脚本，把 `P4-testgen` 吐出的这个数学逻辑，自动翻译成人类的自然语言：

  【Prompt / 需求】：请编写一个 P4 程序，要求在数据面解析 IPv4 头部。当控制面匹配到目的 IP 为 10.0.0.1 这一规则时，触发 Action C，并将报文重定向到端口 2。
  【Response / 答案】：`（给出对应的 main.p4 源码）`
- **训练价值**：这是最高价值的 **“语义-代码对齐数据”**！你完全不需要人工去给代码写注释或写题目。`P4-testgen` 吐出的测试路径，就是这份代码最精准、绝无二义性的“功能说明书”。

~~1.3 初始监督数据集构建 -- 知识问答~~

~~待定：写脚本让大模型生成，用长度检查、困惑度（PPL）粗筛、MinHash LSH 等方法初筛，再人工复检。应生成 2000 条高质量的。~~

1.4 流水线开发

希望实现，用户输入文本，然后大模型生成多个文件，用户（or 模型自动）决定编译/执行，告诉用户执行进度+反馈错误，允许用户互动修改代码并提交作为后续训练样本。以下论文参考：

The Shift to Agentic AI: Evidence From Codex

Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems

Agent-Computer Interfaces Enable Automated Software Engineering

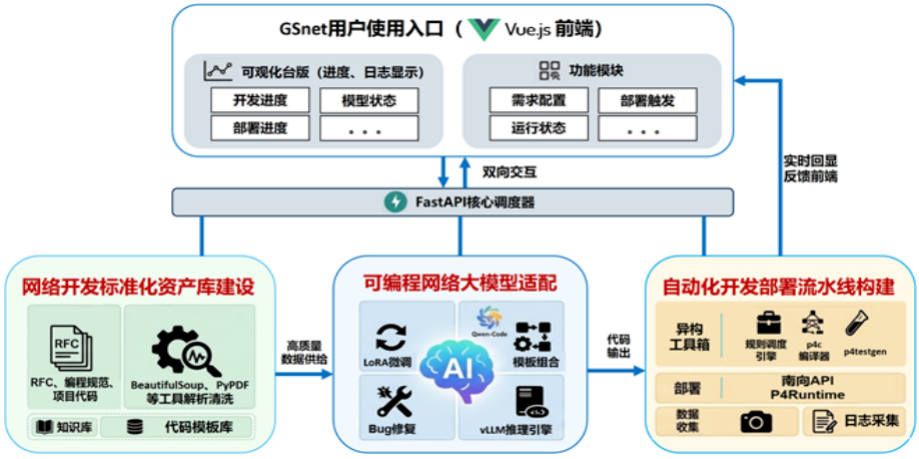
1.5 无监督领域基模型训练

这一阶段，我们要创造的是具有海量知识的基模型，输入“自由度极高”、“规模极大”、“信息密度参差不齐”的数据，采用全量 Loss 令其精准复刻下文。

$$\mathcal{L}_{\text{pretrain}} = -\frac{1}{N}\sum_{i=1}^{N}\log P(x_i \mid x_1, x_2, ..., x_{i-1})$$

starcoder 预训练可参考点：15B 模型，分两阶段（首先的上下文是 4k，然后扩充到 16k），5 个 epoch。我们到底是用全参数微调还是 LoRA？FlashAttention 要不要？仅从精度去衡量取舍。

旧内容



1、 资产库建设

第一步：代码收集 or 手工撰写（可 LLM 辅助）

感知（模型特征提取、模型推理）、测量（sketch、计数器、队列计数、遥测等）、传输、路由、防火墙等功能，每位老师（or 自己的学生）收集大于等于 20 个示例代码，形式：readme（prompt）+代码

主要先构建 BMv2 的，若是收集时发现 eBPF、dpdk、Tofino、FPGA 等平台也可提前收集，收集的代码需通过编译

第二步：提取每种功能中的原语

可能用语法树分析高频子串？

2、 大模型适配（我目前在做的）

调研，LLaMA Factory 环境搭建中（持续训练、后训练）

3、 流水线

编译：p4c 的 bmv2、tofino，外加另外 fpga，这些编译器的打包+统一调用

测试：静态测试、流量样例生成、p4testgen 等

4、 使用测试

略